ESP32 Enclosure Control System

System Documentation & File Reference

February 2026 • OctoPi / ESP32 Integration

---

1. System Overview

This document describes the complete ESP32-based enclosure control system integrated with OctoPrint on a Raspberry Pi. It covers how the three major components work together, where every relevant file lives on the Pi, and how to access and modify them in the future.

The system consists of three main layers:

- ESP32 Firmware (main.cpp) — runs directly on the ESP32 microcontroller, reads sensors, runs a PID control loop, and communicates over USB serial.
- Pi Daemon (enclosure_daemon.py) — a Python Flask service running on the Raspberry Pi that bridges USB serial communication to a local HTTP API.
- OctoPrint Plugin (__init__.py) — a plugin that proxies API commands from the OctoPrint web interface through to the Pi daemon, allowing browser-based control.

> *All three layers must be running simultaneously for the system to work. If any layer is down, the full chain breaks.*

2. Communication Architecture

Data flows through the following chain, from browser to hardware:

```
                    Browser (on laptop/phone)
                              ↕  HTTP via OctoPrint Web UI
              OctoPrint Web Server (port 80 on Pi)
                              ↕  Internal HTTP (localhost)
```

```
OctoPrint Plugin — enclosure_proxy/__init__.py
                    ↕  HTTP API calls to port 8070
Pi Daemon — enclosure_daemon.py (Flask on 127.0.0.1:8070)
                    ↕  USB Serial (/dev/ttyUSB0) at 115200 baud
ESP32 Firmware — main.cpp
                    ↕  Hardware
BME280 Temp Sensor / Heater PWM / Exhaust Fan / Tachometer
```

> *The OctoPrint plugin never talks to the serial port directly. It only makes HTTP requests to the daemon. This keeps serial access centralised and prevents conflicts.*

3. File Locations on the Raspberry Pi

3.1 Quick Reference Table

Path on Pi	Purpose
`/home/pi/enclosure_daemon/enclosure_daemon.py`	Main Pi daemon — Flask HTTP server + serial bridge
`/home/pi/enclosure_daemon/venv/`	Python virtual environment for the daemon
`/etc/systemd/system/enclosure-daemon.service`	Systemd service definition — makes daemon start on boot
`/home/pi/.octoprint/plugins/enclosure_proxy/`	OctoPrint plugin folder (root)
`/home/pi/.octoprint/plugins/enclosure_proxy/__init__.py`	Main plugin Python file — API proxy logic
`/home/pi/.octoprint/plugins/enclosure_proxy/templates/enclosure_proxy_sidebar.jinja2`	OctoPrint sidebar HTML template
`/home/pi/.octoprint/plugins/enclosure_proxy/static/js/enclosure_proxy.js`	Frontend JavaScript — UI logic and API calls
`/home/pi/.octoprint/plugins/enclosure_proxy/static/css/enclosure_proxy.css`	Sidebar styling
`/home/pi/.octoprint/logs/octoprint.log`	OctoPrint log — check here for plugin errors
`/home/pi/.octoprint/config.yaml`	Main OctoPrint configuration file

3.2 ESP32 Firmware

The ESP32 firmware source files are managed separately (not on the Pi) in a PlatformIO or Arduino IDE project. The compiled firmware is flashed directly to the ESP32 over USB. Key source files:

- main.cpp — main program loop, PID controller, command parser, serial protocol

- pinControl.h / pinControl.cpp — HeaterControl and ExhaustFanControl hardware driver classes
- tachometer.h / tachometer.cpp — RPM measurement from the fan tachometer pin

> *To re-flash the ESP32, connect it via USB to your development machine (not the Pi) and upload via PlatformIO or Arduino IDE. The Pi connects to the ESP32 only as a serial client, it does not flash it.*

4. How to Access and Edit Files

4.1 SSH into the Pi

All Pi-side files are edited over SSH. Connect from your laptop:

```
ssh pi@octopi.local
```

Default password is raspberry unless you have changed it. You can also use the Pi's IP address instead of octopi.local.

4.2 Editing the Daemon

```
nano /home/pi/enclosure_daemon/enclosure_daemon.py
```

After saving changes to the daemon, restart its service so the changes take effect:

```
sudo systemctl restart enclosure-daemon
sudo systemctl status enclosure-daemon --no-pager
```

4.3 Editing the OctoPrint Plugin

The plugin has four editable files. Edit each with nano:

```
# Main plugin logic
nano ~/.octoprint/plugins/enclosure_proxy/__init__.py
```

```
# Sidebar HTML
nano
~/.octoprint/plugins/enclosure_proxy/templates/enclosure_proxy_sidebar.jinja2
```

```
# JavaScript UI
nano ~/.octoprint/plugins/enclosure_proxy/static/js/enclosure_proxy.js
```

```
# CSS styling
nano ~/.octoprint/plugins/enclosure_proxy/static/css/enclosure_proxy.css
```

After editing any plugin file, restart OctoPrint:

```
sudo systemctl restart octoprint
```

4.4 Viewing Logs

Plugin and OctoPrint logs are the first place to check when something stops working:

```
# Live log feed
tail -f ~/.octoprint/logs/octoprint.log
```

```
# Last 100 lines filtered for enclosure messages
tail -n 100 ~/.octoprint/logs/octoprint.log | grep -i enclosure
```

```
# Daemon service log
sudo journalctl -u enclosure-daemon -n 50 --no-pager
```

5. How the Pi Daemon Works (enclosure_daemon.py)

The daemon is a Python Flask application that acts as the middleman between the OctoPrint plugin and the ESP32 over USB serial. It runs as a background service and starts automatically on boot.

5.1 Startup Sequence

- connect_serial() is called first. It opens /dev/ttyUSB0 at 115200 baud and waits for the ESP32 to respond to PING commands. It retries indefinitely until the ESP32 is ready.
- A background thread is started running poller(), which sends a GET STATUS command to the ESP32 every 2 seconds and caches the response.
- The Flask HTTP server starts on 127.0.0.1:8070 and begins accepting requests.

5.2 Serial Communication Protocol

Every command sent to the ESP32 uses a numbered prefix so replies can be matched to the correct request:

```
@12 GET STATUS    ← Pi sends, @12 is the message ID
@12 OK TEMP 22.34 RPM 1200 HEATER 0.0 ...   ← ESP32 replies with same ID
```

The send_ser_command() function handles all of this. It increments a global message ID counter, writes the command to serial, then reads lines back until it finds a reply starting with the matching @ID. It waits up to 4 seconds (CMD_TIMEOUT_S) before giving up.

> *The daemon uses a threading Lock (_lock) around all serial access. This prevents the poller thread and an incoming HTTP request from both trying to write to serial simultaneously, which would corrupt the data.*

5.3 HTTP API Endpoints

The daemon exposes the following HTTP endpoints on port 8070, accessible only from localhost:

Path on Pi	Purpose
`GET /ping`	Sends PING to ESP32, returns the PONG reply
`GET /status`	Returns the latest cached status snapshot (updated every 2 seconds by poller)
`POST /setpoint`	Body: {"c": 45.0} — sets target temperature in Celsius
`POST /mode`	Body: {"mode": "AUTO"} — sets mode to OFF, AUTO, or MANUAL

Path on Pi	Purpose
`POST /heater`	Body: {"value": 30.0} — sets heater duty cycle (MANUAL mode only)
`POST /exhaust`	Body: {"value": 80.0} — sets exhaust fan duty cycle (MANUAL mode only)

5.4 Status Caching

Rather than querying the ESP32 on every HTTP request (which would be slow and could cause serial contention), the daemon runs a background poller thread that fetches status every 2 seconds and stores it in the _last_status dictionary. When a GET /status request comes in, the cached value is returned immediately — no serial communication required.

6. How the ESP32 Firmware Works (main.cpp)

The firmware runs a continuous loop on the ESP32 microcontroller. It reads sensors, executes a PID control algorithm, listens for serial commands, and drives the hardware outputs.

6.1 Main Loop Structure

Every iteration of loop() does the following in order:

- Read temperature from the BME280 sensor over I2C. If temperature exceeds MAX_TEMP_C (60°C), enterError() is called immediately — this shuts down the heater, runs the exhaust at 100%, and halts control.
- Read exhaust fan RPM from the tachometer. NaN values (common during spin-up) are replaced with the last valid RPM reading to avoid spurious errors.
- Process any incoming serial characters. Commands are accumulated character-by-character into rxLine. When a newline is received, handleCommandLine() is called.
- If control is enabled (AUTO mode), run one iteration of the PID algorithm and update hardware outputs.
- Short 10ms delay to prevent CPU hogging.

6.2 The PID Controller

The PID controller runs in AUTO mode and calculates how much heating or cooling is needed to reach the setpoint temperature. The three terms are:

- Proportional (P = KP × error) — Reacts proportionally to how far the temperature is from the setpoint. KP = 25.0.
- Integral (I = KI × accumulated error) — Slowly corrects for persistent small offsets. KI = 0.15. Anti-windup is applied: the integral only accumulates when error is within 3°C, and is clamped to ±20.

- Derivative (D = KD × rate of change) — Acts as a brake to prevent overshoot. KD = 40.0.

The combined PID output is mapped to hardware:

- Positive output (heating needed) — heater duty increases from MIN_HEATER_OUTPUT (5%) to 100%. Exhaust fan is off.
- Negative output (cooling needed) — exhaust fan duty increases from MIN_COOLING_OUTPUT (25%) to 100%. Heater is off.
- Near zero (within DEADBAND of 0.2) — both outputs are shut down.

6.3 Control Modes

Mode	Behaviour
`OFF`	PID is disabled. Both heater and exhaust are shut down. Outputs stay at 0. System is idle but still responds to serial commands.
`AUTO`	PID runs every 500ms. The firmware automatically drives heater or exhaust based on the difference between the measured temperature and the setpoint. Manual heater/exhaust commands are ignored.
`MANUAL`	PID is disabled. The operator directly controls heater duty and exhaust duty via SET HEATER and SET EXHAUST commands. Outputs hold their last set value until changed. Switching to MANUAL resets both outputs to 0 as a safe starting point.

6.4 Serial Command Reference

The ESP32 accepts the following commands over USB serial (sent by the daemon):

Path on Pi	Purpose
`PING`	Health check. Returns: OK PONG
`GET STATUS`	Returns full telemetry: TEMP, RPM, HEATER, EXHAUST, SETPOINT, MODE, CONTROL, SAFETY
`SET MODE OFF`	Disables control, shuts down all outputs
`SET MODE AUTO`	Enables PID controller. Rejected if a safety fault is active.
`SET MODE MANUAL`	Disables PID, resets outputs to 0, enables manual control commands
`SET SETPOINT {c}`	Sets target temperature in °C. Allowed range: 10.0–50.0°C
`SET HEATER {duty}`	Sets heater PWM duty (0–100). Only works in MANUAL mode.

Path on Pi	Purpose
`SET EXHAUST {duty}`	Sets exhaust fan duty (0–100). Only works in MANUAL mode.
`RESET_FAULT`	Clears a safety fault so AUTO mode can be re-enabled

6.5 Safety Fault System

If the temperature exceeds MAX_TEMP_C (60°C), the firmware calls enterError(). This:

- Shuts down the heater immediately
- Runs the exhaust fan at 100% for active cooling
- Sets safetyTripped = true, which blocks AUTO mode from being re-enabled
- Sends an EVT FAULT message over serial, which the daemon can log

To recover from a safety fault, the operator must first resolve the overtemperature condition, then send the RESET_FAULT command (e.g. via the OctoPrint plugin). AUTO mode can then be re-enabled.

7. How the OctoPrint Plugin Works

The OctoPrint plugin (__init__.py) is a SimpleApiPlugin that exposes HTTP API endpoints within OctoPrint. Its only job is to receive requests from the browser-side JavaScript and forward them to the daemon running on port 8070.

7.1 Plugin Mixins Used

- SimpleApiPlugin — creates the GET and POST API endpoints at /api/plugin/enclosure_proxy
- StartupPlugin — runs on_after_startup() when OctoPrint finishes loading, used for logging
- TemplatePlugin — registers the sidebar HTML template so OctoPrint includes it in the web UI
- AssetPlugin — registers the JavaScript and CSS files so OctoPrint serves them to the browser

7.2 API Endpoints (browser to plugin)

The browser communicates with the plugin through two OctoPrint routes:

```
GET  /api/plugin/enclosure_proxy
```

Returns the current status JSON fetched from the daemon's /status endpoint.

```
POST /api/plugin/enclosure_proxy
```

Accepts a JSON body with a command field and optional parameters. Valid commands: setpoint, mode, heater, exhaust. The plugin maps each command to the appropriate daemon POST endpoint.

> *All requests from the browser must include the OctoPrint API key in the X-Api-Key header. The plugin's API key can be found in OctoPrint Settings → Application Keys.*

7.3 Sidebar UI

The JavaScript file (enclosure_proxy.js) polls GET /api/plugin/enclosure_proxy every 2 seconds and updates the displayed temperature, RPM, and mode. When the user clicks buttons or moves sliders, it sends POST commands to the plugin, which forwards them to the daemon, which writes to the ESP32 serial port.

8. Systemd Services

Both the daemon and OctoPrint are managed by systemd and start automatically on boot.

8.1 Useful Service Commands

Path on Pi	Purpose
`sudo systemctl status enclosure-daemon`	Check if daemon is running
`sudo systemctl restart enclosure-daemon`	Restart daemon (after code changes)
`sudo systemctl stop enclosure-daemon`	Stop daemon
`sudo systemctl start enclosure-daemon`	Start daemon
`sudo systemctl status octoprint`	Check if OctoPrint is running
`sudo systemctl restart octoprint`	Restart OctoPrint (after plugin changes)

8.2 Daemon Service File

The systemd service file that defines how the daemon starts is located at:

```
/etc/systemd/system/enclosure-daemon.service
```

To edit it:

```
sudo nano /etc/systemd/system/enclosure-daemon.service
sudo systemctl daemon-reload   # required after editing the service file
sudo systemctl restart enclosure-daemon
```

9. Troubleshooting Quick Reference

Symptom	What to check
OctoPrint sidebar not visible	Restart OctoPrint. Check ~/.octoprint/logs/octoprint.log for plugin errors.
API returns 404	Plugin not loaded. Check log for import errors. Confirm __init__.py syntax is valid.
API returns 500	Plugin loaded but code is crashing. Check log for Python traceback.
API returns 502	Plugin is running but cannot reach the daemon. Run: systemctl status enclosure-daemon
Daemon won't start	Check: sudo journalctl -u enclosure-daemon -n 50. Usually a wrong path or Python error in the daemon file.

Symptom	What to check
Daemon starts but status shows ok:false	ESP32 is not responding. Check USB cable, check /dev/ttyUSB0 exists, try unplugging and replugging the ESP32.
Temperature not updating	Daemon poller may have stalled. Restart enclosure-daemon service. Check if BME280 is connected properly.
Heater/exhaust not responding to MANUAL commands	Confirm mode is MANUAL first. Check that SET MODE MANUAL was acknowledged.
Safety fault — cannot enable AUTO	Temperature exceeded 60°C. Resolve the hardware issue, then send RESET_FAULT command via the plugin.